

OpenRath: Session-Centered Runtime State for Agent Systems

Fukang Wen, Zhijie Wang, Ruilin Xu

ABSTRACT

Modern agent systems often suffer from fragmented runtime state: transcripts, tool effects, memory events, workspace placement, branch provenance, and replay evidence are recorded separately and become difficult to inspect or reproduce. OpenRath addresses this issue with a PyTorch-like programming model for multi-agent, multi-session systems. The analogy concerns the role of a central first-class runtime abstraction, not tensor computation. Its core abstraction is Session, the runtime value passed between agents and workflows. A Session is branchable, inspectable, replayable, backend-aware, and composable. It records conversation chunks, sandbox placement, lineage metadata, token usage, pending work, and tool evidence, while defining where memory interactions enter the runtime record. Since this state is carried by the same value used in program execution, fork, merge, and replay become explicit runtime operations rather than states reconstructed from external traces. OpenRath further defines Sandbox, Tool, Agent, Memory, Workflow, and Selector, with Selector turning control flow into runtime-routed decisions. This report presents the programming model, architecture, audited milestones, and evidence protocol. Its claims are limited to controlled runtime properties, while broad quantitative comparisons, live-provider quality, optional-backend availability, and memory quality are left for follow-on evaluation. The central thesis is that Session provides agent systems with a first-class runtime value for auditable composition.

About this document

This is a **Reviewed Preprint**: a third-party paper that llmXive has *auto-reviewed* (with its LLM review panel) but *never modified*. The original manuscript follows this cover page **exactly as published** by its authors — llmXive re-typesets nothing and claims no authorship.

Provenance. Ingested by llmXive on 2026-07-03 — scraped from arXiv; via llmXive dashboard, GitHub issue #357; submitted by github-actions[bot].

Source. <https://arxiv.org/abs/2606.19409>

About llmXive. llmXive is an automated platform for open scientific discovery. Its specialist LLM agents advance their own ideas from brainstorm to peer-reviewed paper, and they also review notable third-party papers — drawn from the most-downloaded papers on Hugging Face and from submissions by human contributors. Every submitted paper is reviewed by the LLM panel

and used to seed a new llmXive follow-up study. This paper is one such third-party work: llmXive reviewed it but did not write it. Learn more at <https://github.com/ContextLab/llmXive>.

Automated review. See the accompanying [peer-review-llmxive.pdf](#) for the full reviewer report.

llmXive follow-up. A separate llmXive study building on this work: [PROJ-927-llmxive-follow-up-extending-openrath-ses](#).

OpenRath: Session-Centered Runtime State for Agent Systems

Fukang Wen^{*,†}, Zhijie Wang^{*}, Ruilin Xu[†]

Abstract

Modern agent systems often suffer from fragmented runtime state: transcripts, tool effects, memory events, workspace placement, branch provenance, and replay evidence are recorded separately and become difficult to inspect or reproduce. OpenRath addresses this issue with a PyTorch-like programming model for multi-agent, multi-session systems. The analogy concerns the role of a central first-class runtime abstraction, not tensor computation. Its core abstraction is **Session**, the runtime value passed between agents and workflows.

A **Session** is branchable, inspectable, replayable, backend-aware, and composable. It records conversation chunks, sandbox placement, lineage metadata, token usage, pending work, and tool evidence, while defining where memory interactions enter the runtime record. Since this state is carried by the same value used in program execution, fork, merge, and replay become explicit runtime operations rather than states reconstructed from external traces.

OpenRath further defines **Sandbox**, **Tool**, **Agent**, **Memory**, **Workflow**, and **Selector**, with **Selector** turning control flow into runtime-routed decisions. This report presents the programming model, architecture, audited milestones, and evidence protocol. Its claims are limited to controlled runtime properties, while broad quantitative comparisons, live-provider quality, optional-backend availability, and memory quality are left for follow-on evaluation. The central thesis is that **Session** provides agent systems with a first-class runtime value for auditable composition.

1 Introduction

1.1 Problem Framing

Consider a long agent run: it plans, forks a branch to test an approach, calls tools, edits files inside a sandbox, recalls memory, compresses context, and eventually returns a correct answer. The natural audit questions are straightforward: which branch produced the final result, which tool modified which file, which memory item was recalled or committed, and which evidence was removed during compression? In many systems, the run cannot answer these questions. The final output may be correct, but the runtime state that produced it has been fragmented across side channels.

Modern agent applications increasingly resemble runtime systems rather than isolated conversations. A simple loop that interleaves reasoning and acting [1]—appending messages, calling a model,

^{*}Equal contribution: Fukang Wen and Zhijie Wang.

[†]Corresponding authors: Fukang Wen and Ruilin Xu. Contact: wfk25@mails.tsinghua.edu.cn.

executing tools, and appending observations—remains a useful pattern for a single assistant. Yet this loop becomes a weak state boundary once work is distributed across roles, tools, memory stores, sandboxes, branches, and resumed executions.

OpenRath identifies this fragmentation as a hidden-runtime-state problem. A message list preserves the conversational surface, but it typically does not expose role provenance, abandoned branches, tool placement, workspace effects, memory recall or commit events, or evidence discarded by compression. Without lineage, tool evidence, sandbox metadata, and usage records, a final answer is difficult to audit once the model, provider, workspace, or prompt changes.

OpenRath therefore starts from the runtime-state boundary rather than from the number of agents in a loop. Its goal is to make the state passed between agents explicit enough to support composition, inspection, branching, merging, persistence, and evaluation. This is why the system is organized around **Session**. A **Session** is not merely chat history; it is the first-class runtime value that carries the evidence required to continue, review, and explain agent work.

1.2 Central Claim

The central claim is that agent systems benefit from a first-class runtime state, and OpenRath proposes **Session** as that state. To clarify the role of this object, OpenRath adopts a PyTorch-inspired programming model [2]: not PyTorch’s tensor mathematics, but its architectural interface for composable computation. In that interface, a central value flows through reusable modules, modules expose a uniform **forward** mapping, placement is made explicit through operations such as `tensor.to(device)`, and persistent module state is represented by parameters. OpenRath adapts this pattern to agent runtimes. **Session** plays the role of the flowing value; **Agent** is a reusable transformation similar in role to a layer; **Workflow** is a compositional container; both follow a `forward(session) -> session` contract; placement is expressed as `session.to(backend)`; and **Memory** is treated as an agent-bound persistent state plane rather than hidden prompt text.

Because each transformation preserves the **Session** `-> Session` shape, agents can be nested into workflows without introducing a separate runtime state format. Composition, branching, merging, handoff, and replay operate on ordinary program values rather than on state reconstructed from controller logs. The analogy is architectural rather than literal: the claim is not that agent systems are neural networks, but that agent runtimes need a stable flowing value, reusable transformations behind a uniform interface, explicit placement, persistent state, and inspectable evidence. The compact vocabulary that realizes this design—**Agent**, **Workflow**, **Tool**, **Memory**, and **Sandbox**—is developed in the programming model that follows.

Why make **Session** the runtime boundary, rather than placing this state inside a graph runtime’s node state or a tracing system’s spans? These layers serve different primary readers. Graph state records where execution is in a control flow, so that a run can resume, replay, or fork from checkpoints. Trace spans record what was observed during execution, such as model calls, tool calls, handoffs, guardrails, and other monitored events. Neither representation is designed to be the ordinary program value that agents themselves fork, merge, hand off, and replay. A trace is written for observers; a graph checkpoint is written for schedulers. A **Session** is written for the agent program: it is the live value passed through the program, and evidence is attached to that value rather than reconstructed from a side channel. OpenRath’s design hypothesis is that multi-agent systems remain more inspectable as they scale when runtime state is placed where the program already flows, rather than beside it.

Table 1: Three runtime records, three readers. OpenRath’s **Session** is the value written for the agent program itself, which is why fork, merge, and replay are first-class rather than reconstructed.

Record	Written for	What it primarily holds
Graph checkpoint	The scheduler	Where execution is in the control flow, so a run can resume or time-travel.
Trace span	The observer	What was observed during a run, for monitoring and debugging after the fact.
Session (OpenRath)	The agent program	The live value agents fork, merge, hand off, and replay; lineage, tools, placement, memory, and usage travel with it.

1.3 Ecosystem Positioning

The ecosystem positioning is intentionally narrow. Agent infrastructure is moving toward durable execution, richer tracing, standardized tool/data protocols, and real-environment evaluation; representative systems include AutoGen [3], LangGraph [4], the OpenAI Agents SDK [5], and MCP [6, 7]. OpenRath complements those layers by working at a different boundary: the runtime value that carries their effects. A graph runtime can schedule work, a tracing system can observe spans, MCP can expose tools, and a sandbox can run commands; OpenRath asks how those effects become one branchable, inspectable, replayable state object that agent programs can pass between agents and workflows. Related Work develops this layer by layer; here it is enough to fix the boundary.

OpenRath is neither a universal substitute for graph runtimes, tracing systems, MCP servers, sandbox providers, or benchmark harnesses, nor a thin wrapper around any one of them. The intended role is connective: a **Session** is the object that can be scheduled, traced, dispatched, persisted, forked, merged, compressed, and reviewed without forcing each layer to invent its own incompatible representation of agent state. The central claim is smaller and more defensible: multi-agent systems need a first-class runtime state, and **Session** is OpenRath’s candidate for that state.

1.4 Contributions

This report makes four technical contributions to the runtime-state boundary for agent systems.

1. **A session-centered runtime dataflow.** OpenRath treats **Session** as the value that moves through the agent runtime, so conversation chunks, placement, lineage, usage, pending work, tool evidence, and memory-boundary records are represented as one inspectable flow rather than separate controller bookkeeping.
2. **A PyTorch-like object vocabulary for agent programs.** The framework organizes agent programs around **Session**, **Sandbox**, **Tool**, **Agent**, **Memory**, **Workflow**, and **Selector**. Each object has a narrow runtime boundary while preserving the same **Session** -> **Session** shape, including runtime-routed control flow.
3. **Backend-aware boundaries for tools and memory.** OpenRath separates runtime state from the execution backend that runs tools and the memory backend that persists recallable state. This lets local execution, optional OpenSandbox placement, MCP-style tools, and memory services participate in one session-centered model as their evidence packets are verified.

4. **An audit-first release protocol.** The report maps claims to packets: lineage export, local sandbox execution, workflow transcript, focused tests, visual QA, claim ledger, and a memory source audit. Broad benchmark superiority, human preference results, and cross-system leaderboard claims are reserved for follow-on quantitative evaluation.

1.5 Runtime State at a Glance

The core visual distinction is simple. A loop-centered agent treats messages, tool logs, memory updates, usage, workspace effects, and branch provenance as side channels around the loop. OpenRath moves those effects into one typed runtime value. The same **Session** can be passed to agents, forked for independent work, merged after review, persisted as evidence, and replayed with explicit backend boundaries. OpenRath does not replace tool protocols, sandbox providers, memory stores, tracing systems, or graph schedulers. It records their effects in a session object that can move through the program as branchable, inspectable, and replayable runtime state.

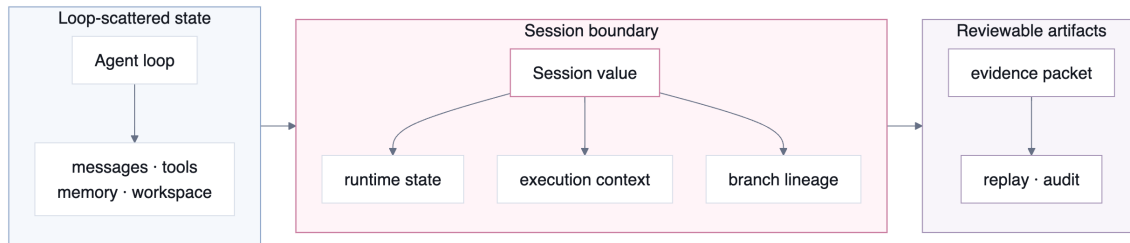


Figure 1: OpenRath’s core boundary: side-channel state around an agent loop is promoted into a branchable **Session** value that can produce release evidence artifacts.

2 Related Work

The agent ecosystem is converging on a specialized runtime stack: reasoning-and-acting methods, multi-agent frameworks, durable graph runtimes, tracing SDKs, tool/data protocols, real-environment benchmarks, and provenance standards each own one layer. The open design question is the *crossing object*—what state can move through these layers while keeping conversation, lineage, placement, tool effects, memory, and artifacts together. We survey these areas by that question and, for each, mark the distinction from OpenRath’s **Session**.

2.1 Tool-Using and Acting Agents

Chain-of-thought prompting and self-consistency elicit and stabilize intermediate reasoning at inference time [8, 9]. ReAct interleaves reasoning with environment-directed actions [1], and MRKL combines a model with external knowledge and discrete reasoning modules [10]. Tool use itself is taught or routed by Toolformer [11], HuggingGPT’s controller over expert models [12], and Gorilla’s retrieval-grounded API calls [13], and is studied at scale by ToolLLM, API-Bank, and ToolAlpaca [14–16]; Tree of Thoughts adds deliberate search with backtracking [17], an inference-time search rather than a persistent, replayable branch. These works advance *how* a model reasons and acts; OpenRath is complementary, making the runtime state those actions produce—lineage, tool evidence, placement—a first-class value.

2.2 Multi-Agent Frameworks

AutoGen frames applications as multi-agent conversations [3], CAMEL studies role-playing communicative agents [18], MetaGPT encodes standardized operating procedures into a collaboration pipeline [19], ChatDev runs a virtual software company over a chat chain [20], and AgentVerse studies dynamic group collaboration and emergent behavior [21]. These contribute orchestration patterns. The distinction is one of object boundary: where MetaGPT’s SOP governs *which role acts when*, OpenRath governs *what value the roles pass*, so multi-agent composition needs no second, framework-private state object.

2.3 Runtime State, Protocols, and Observability

LangGraph exposes checkpointed graph state with history and time travel to replay or fork from a checkpoint [4, 22]; an OpenRath **Session**, by contrast, is the value the program itself passes and forks, not a scheduler checkpoint. The OpenAI Agents SDK records traces and spans over generations, tool calls, handoffs, and guardrails and composes agents from those parts [5, 23], and OpenTelemetry treats spans as an observer-facing signal [24]; traces describe *what was observed* after the fact, whereas **Session** is written for the program, so its evidence is the value itself. Connectivity is standardized by the Model Context Protocol [6, 7] and interface descriptions such as OpenAPI [25]. The dataflow-runtime analogy is instructive: TensorFlow represents computation and shared state as a graph [26], while OpenRath keeps the value imperative and lets lineage, placement, and evidence travel with it. Table 2 summarizes the object-boundary question each layer leaves open.

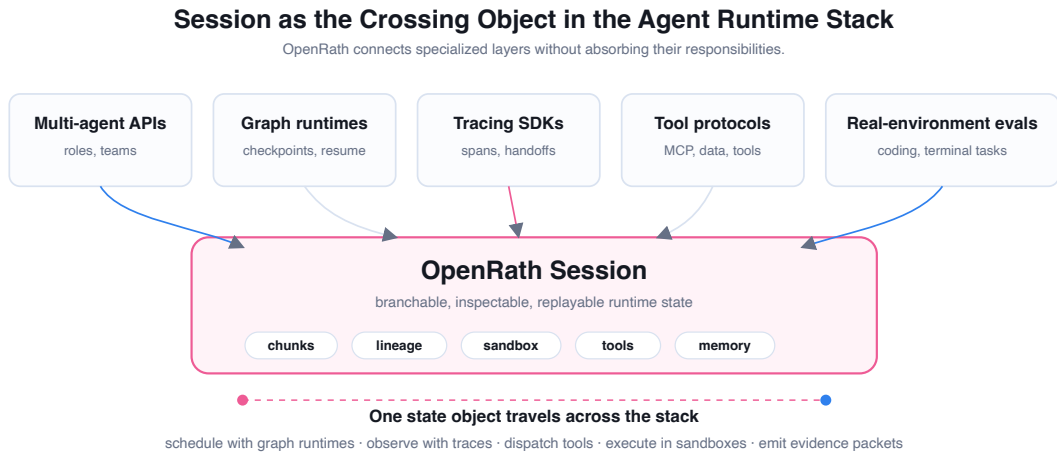


Figure 2: OpenRath’s ecosystem role is a crossing-object boundary. It can work with specialized agent APIs, graph runtimes, tracing SDKs, tool protocols, sandbox providers, and evaluation harnesses by making their effects visible in one **Session**.

Table 2: Runtime-stack trends and OpenRath’s intended boundary.

Trend	What became first-class	Remaining object-boundary question
Multi-agent APIs	Agent roles, teams, group chats, and workflow patterns.	What state moves between roles besides transcript text?
Durable graph runtimes	Checkpoints, thread state, state history, resume, and time travel.	What session-level evidence should a graph node carry?
Tracing SDKs	Model spans, tool calls, handoffs, guardrails, and custom events.	What runtime value should traces attach to and replay from?
Tool/data protocols	Standardized access to external tools, data, and workflows.	How do external effects return as lineage, artifacts, and backend evidence?
Real-environment benchmarks	Repository tasks, terminal tasks, GUI/CLI environments, tests, and scored outcomes.	Can reviewers inspect how the outcome was produced?

2.4 Memory and Retrieval

A large body of work studies what an agent should remember and how it should retrieve it. At the agent level, Reflexion converts feedback from failed attempts into natural-language reflections held in an episodic buffer, so later attempts improve [27]; Generative Agents maintain a long-running memory stream that is retrieved, reflected upon, and compiled into plans [28]; and MemGPT adopts the operating-system idea of a memory hierarchy, paging information between a bounded context window and external storage to sustain long interactions [29]. Voyager carries this toward lifelong skill acquisition, growing a reusable library of verified behaviors from environment feedback [30]. OpenRath does not propose a new memory model; it simply makes memory operations session-visible, so recall and commit are recorded as explicit runtime events on `Session` rather than hidden inside the prompt.

2.5 Agent Benchmarks and Environments

Interactive evaluation spans AgentBench [31] and τ -bench’s tool-agent-user setting with database-state checks [32]; software engineering through SWE-bench [33], SWE-agent’s agent-computer interface [34], and the human-filtered SWE-bench Verified subset [35]; terminals through TerminalBench, TerminalWorld, and task-alignment studies [36–38]; and web, desktop, and embodied settings including WebArena, VisualWebArena, WorkArena, OSWorld, WebShop, Mind2Web, ALFWorld, ScienceWorld, GAIA, and TheAgentCompany [39–48]. These score outcomes inside realistic environments; OpenRath’s complementary question is whether the trajectory that produced an outcome is inspectable and replayable—a precondition for trustworthy scoring.

3 Background and Motivation

Section 1 framed the gap with a single run, and the related work above situated OpenRath among adjacent agent systems—tool-using and acting agents, multi-agent frameworks, runtime/observability layers, memory and retrieval, evaluation environments, and provenance standards. This section states the gap that runs underneath all of them as a general property of multi-agent work. The loop boundary that suffices for one assistant becomes a weak state boundary the moment an

application branches across roles, tools, memory, files, sandboxes, and resumed runs. The transcript still shows a final answer, but the runtime path that produced it is spread across controller code, tool logs, memory stores, workspace state, and provider traces.

This pressure is becoming more visible as agent products move from demos to longer-running workflows. Once an agent edits a repository, calls external tools, resumes after interruption, or routes work through multiple roles, the state boundary becomes an engineering contract rather than an implementation detail. Users and reviewers need to ask ordinary runtime questions: what was the input state, what changed, which backend performed the change, which memory or artifact influenced the decision, and how can the run be resumed or replayed? A framework that cannot answer those questions may still produce a plausible response, but it cannot easily support release review, debugging, audit, or systematic evaluation.

This hidden state is the central motivation for OpenRath. Multi-agent work is not merely one conversation with more roles. It naturally creates multiple runtime paths: one branch gathers context, another edits or tests an artifact, another validates evidence, and another compresses the result. If every intermediate step is placed into one shared transcript, later agents inherit too much noise. If intermediate work is hidden in controller state, reviewers cannot reconstruct which branch produced a claim, what memory was recalled, which sandbox touched a file, or what evidence was discarded during compression.

OpenRath treats branchability as a property of runtime state rather than as a controller-side convention. A branch should inherit the portion of parent context required for independent work, accumulate local evidence during execution, and merge useful results back without erasing provenance. The object being branched is therefore **Session**: the runtime value that flows through agents, tools, memory-boundary operations, sandbox placement, compressors, and workflows.

This boundary is the foundation for the remainder of the report. The next section makes it concrete through a compact object vocabulary centered on **Session**, the value that every other component reads, transforms, annotates, or passes forward.

4 OpenRath Programming Model

OpenRath keeps the programming model small on purpose. The core rule is that runtime components transform or annotate **Session**; they should not each invent a private transcript, placement record, tool log, memory format, or workflow state. This rule is what makes the PyTorch analogy useful: the analogy is not about tensor math, but about one value flowing through reusable transformations with explicit placement and persistent state boundaries.

Table 3: The compact OpenRath object vocabulary.

Object	Runtime boundary
Session	Flowing runtime value for chunks, placement, lineage, usage, pending work, tool evidence, and memory evidence when enabled.
Agent	Reusable Session -> Session transformation with local prompt, provider, tools, and memory policy.
Tool	Model-visible callable operation backed by schema validation, session context, sandbox dispatch, and returned evidence.
Sandbox	Placement boundary for file, command, code, and external tool execution.
Memory	Intended persistent-state plane for recall and commit across runs, kept separate from prompt text.
Workflow	Composition surface for agents, tools, branches, compression, memory, and child workflows.
Selector	Runtime router over self-describing workflows: it reads the current session and picks the next workflow, so dynamic control flow stays explicit instead of hard-coded.

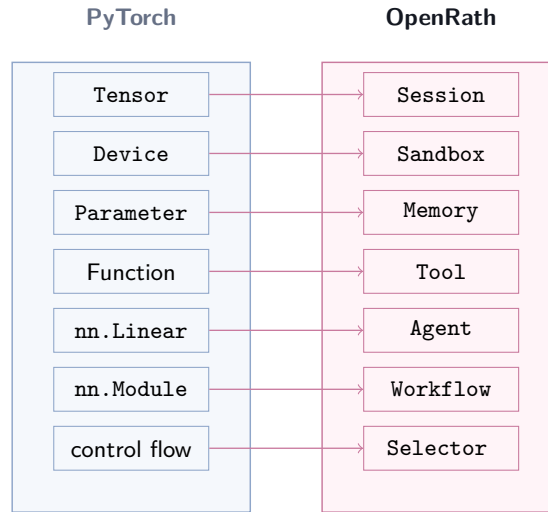


Figure 3: The PyTorch lens. Each agent-runtime concern maps onto one OpenRath object, with **Session** as the flowing value (the tensor of the runtime) and **Selector** routing control flow at run time. The mapping is a teaching device, not a claim that agent systems are neural networks.

The most important design choice is what each object does not own. An **Agent** does not own the entire conversation graph; lineage belongs to **Session**. A **Tool** does not own placement; it executes through the active sandbox. A **Workflow** does not create a separate orchestration state; it composes transformations over sessions. **Memory** does not become hidden prompt text; recall and commit should remain visible runtime events. These separations keep the system inspectable when a run becomes multi-agent, multi-branch, and multi-backend.

The tool boundary illustrates the pattern. A flow-level tool exposes a name, description, and JSON schema to the model, while its Python call receives the active **Session** and validated arguments. Built-in tools can then create backend payloads for file, command, code, or MCP-like execution without changing the model-visible contract. The same principle applies to workflows: a workflow may fork a session, call an agent, validate in a sandbox, compress context, and return a new session, but the evidence remains attached to the shared runtime value.

Control flow follows the same discipline through **Selector**. Rather than hard-coding which agent or workflow runs next, a **Selector** reads the current **Session** and routes to one of several self-

describing workflows, returning an empty workflow when the task is done. This keeps branching and looping over agents as ordinary, inspectable runtime decisions: the routing choice becomes part of the session record instead of vanishing into controller code. It is also where OpenRath departs from a static workflow graph—the next step is decided at runtime from session state, yet every decision still flows through one value.

Memory is described with a deliberately bounded claim. OpenRath provides local memory with lexical recall, optional embeddings, and an optional external backend, exposed through agent-level recall and commit operations so that remembering and recalling stay visible runtime events rather than hidden prompt text. What this report does not claim is retrieval *quality*: how well a given corpus, embedding choice, and commit policy serve a task is an empirical question left to a follow-on evaluation. The programming model reserves the correct boundary—memory as a session-visible persistent plane—without asserting that every quality and backend trade-off has been measured.

5 Runtime Architecture

The runtime architecture answers one question: how does a **Session** remain inspectable as it moves through agents, tools, sandboxes, branches, and stored artifacts? OpenRath uses a small lifecycle rather than a separate runtime object for every phase. A session is created from user or agent context, placed on an execution backend when needed, transformed by agents or workflows, branched for parallel work, merged after review, persisted for replay, and released when sandbox resources are no longer owned.

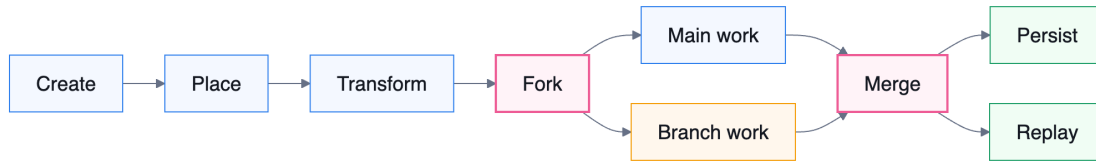


Figure 4: Session lifecycle as a single runtime value: the same object is placed, transformed, branched, merged, persisted, and replayed rather than replaced by a separate orchestration state.

Table 4: Runtime path of an OpenRath session.

Phase	What becomes auditable
Create	Initial user text, agent prompts, role, and ordered chunk state.
Place	Backend intent from <code>Session.to(...)</code> and the workspace where tools can run.
Transform	Model calls, tool requests, results, errors, compressors, workflow steps, and usage.
Branch	Parent sessions, fork , detach , merge , branch provenance, and merge inputs.
Persist	Replayable chunks, lineage JSONL rows, usage, and source evidence.
Release	Sandbox handle ownership and backend lifetime.

Branching is the point where a transcript becomes a graph. **fork** duplicates state while preserving the parent relation; **detach** starts a new lineage root from copied content; **merge** joins compatible sessions and records both parents. In the current implementation, merge compatibility includes sandbox compatibility: sessions must share a live sandbox handle or target the same unbound backend. This makes placement part of the runtime graph rather than an external execution detail.

Tool execution follows a layered path. The model sees FlowToolCall schemas. The session loop combines built-in and user tools, sends schemas to the provider, resolves returned tool calls by

name, validates arguments, and invokes the selected tool with the active session. When a tool needs side effects, it dispatches a backend payload through the session’s sandbox. Malformed arguments, unknown tools, exceptions, and successful results all become tool-result chunks rather than disappearing into controller flow.

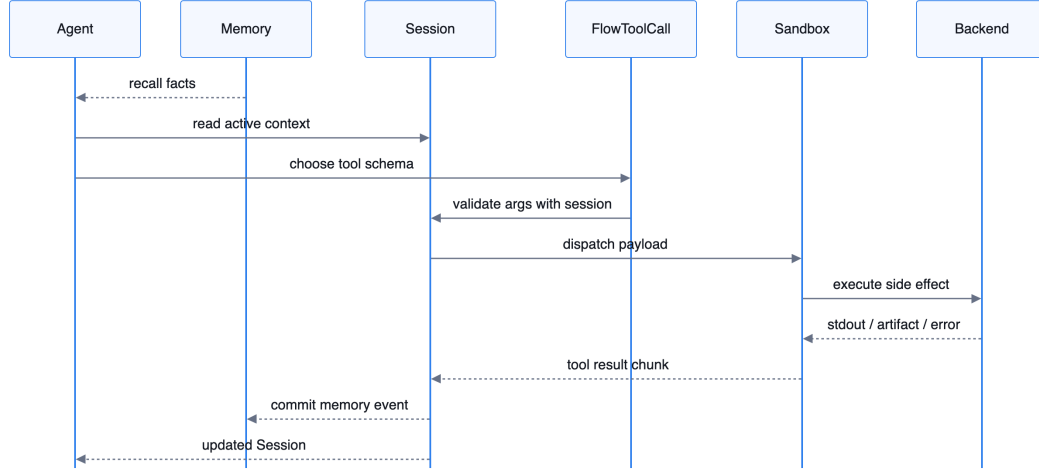


Figure 5: Tool execution boundary: schemas are visible to the model, side effects run through the session’s sandbox and backend, and results return as session evidence.

Table 5: Backend boundary used by tool execution.

Boundary	Owner and role
Placement intent	Session stores the backend name and opening spec before execution.
Resource lifetime	A sandbox handle shares or releases live execution resources across branches.
Capability claim	The backend class advertises isolation level and supported tool payloads.
Concrete execution	The backend instance runs local, OpenSandbox, or future backend operations.
Evidence return	The session loop appends results or errors to the session stream.

Persistence and replay close the loop. A running session can append rows to its session JSONL store; lineage export can project sessions into plain JSONL rows containing identifiers, parent identifiers, lineage operator, lineage kind, chunk count, and cumulative usage. The format is intentionally boring: it can be inspected with command-line tools, attached to release evidence, and converted into diagrams later. This is the architectural through-line of the report: OpenRath makes agent work easier to evaluate because conversation, tools, placement, lineage, usage, and replay artifacts are carried by the same session-centered runtime path.

6 Multi-Agent Multi-Session Design

Multi-agent design in OpenRath is intentionally small: an agent is a reusable layer, a workflow is a reusable composition, and the moving runtime value is still Session. This avoids a common failure mode in agent systems, where a single-agent API works cleanly but the multi-agent version introduces a new shared mutable object, a hidden message bus, or a controller-only trace.

The engineering examples use this shape for lead-engineer, specialist, and QA roles; the research examples use the same shape for literature, reproduction, compression, and output stages. The domain-specific roles differ, but the runtime contract does not. This is the point of the design: a

workflow can grow from a script into a nested agent team without replacing the object that carries evidence, placement, lineage, usage, and replay state.

This report therefore treats multi-agent capability as a runtime-state claim, not as a claim that every workflow is already a measured benchmark result. Current evidence verifies deterministic lineage export, local sandbox packets, workflow transcripts, focused tests, and layout review. Larger claims about parallel branch scheduling, merge quality, memory quality, and task-level leaderboards remain scoped to follow-on evaluation.

From Multi-Session Runtime to Multi-Agent Workflow

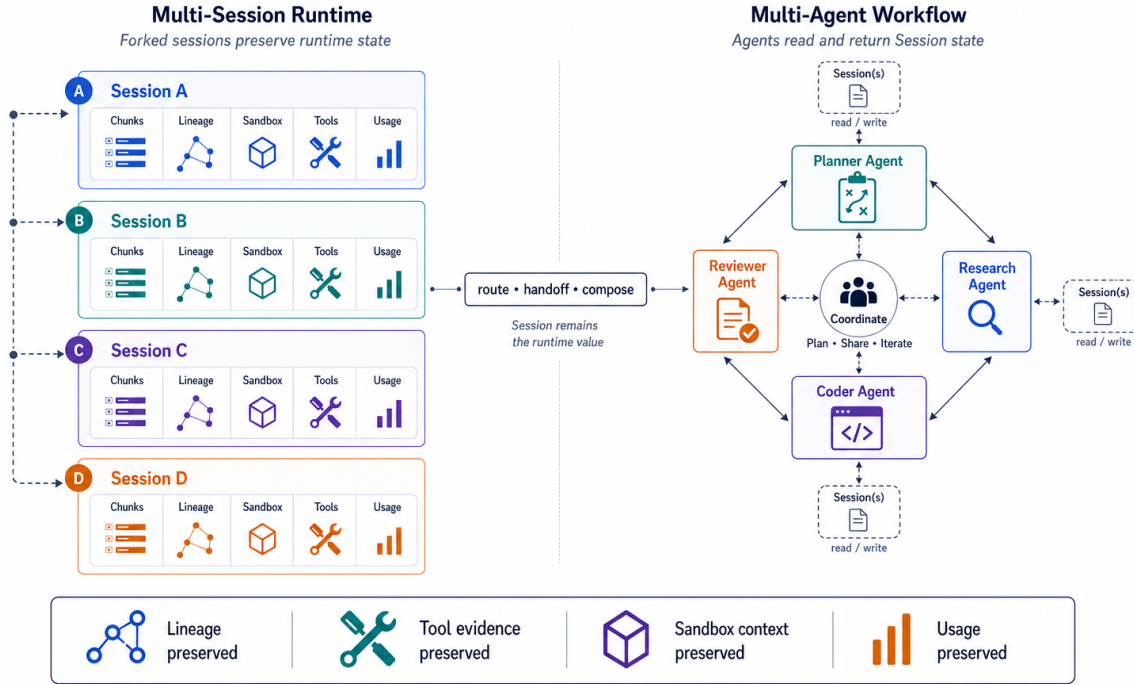


Figure 6: Multi-session runtime and multi-agent workflow share the same boundary: agents route, hand off, and compose work by reading and returning **Session** state rather than introducing a second runtime object.

Table 6: Multi-agent composition without introducing a second runtime state.

Pattern	How it composes	What stays inspectable
One agent, many sessions	The same agent parameters are applied to fresh, forked, resumed, or sandbox-bound sessions.	Prompt layer stays stable while conversation, placement, lineage, and usage live on the session.
Many agents, one state	Specialist agents each consume and return Session ; workflows pass the returned value forward.	Intermediate state can be inspected after planning, tool use, compression, QA, or final synthesis.
Nested workflows	A child workflow hides internal agent structure behind forward(session) .	Parent workflows see one returned session instead of a private orchestration format.
Control surfaces	Chunks, tool results, lineage fields, usage counters, persistence files, and callbacks remain tied to the session.	Failures, budget crossings, branch provenance, and interrupted runs become reviewable artifacts.

7 Implementation Milestones

OpenRath is a working implementation, not an architecture sketch. It is distributed as a Python package whose modules realize the objects of the preceding sections: a session core; an execution-backend layer; the flow layer of tools, agents, workflows, and compressors; an LLM-provider layer; and persistence with lineage export. This report is written against an audited snapshot of that codebase, which we treat as adequate for technical review rather than as a tagged archival release. Table 7 records which surfaces the current implementation substantiates and where its claims are deliberately bounded.

Table 7: Implementation surface and claim status.

Surface	Status
Session core	Implemented runtime value: ordered chunks, branching via fork , detach , and merge , usage accounting, and JSONL lineage export, exercised by focused tests.
Backend placement	Local execution implemented and verified; OpenSandbox available as an optional backend, unconfigured in this environment.
Tool layer	Implemented boundary: model-visible schemas with backend-dispatched side effects returned to the session, with custom-tool and MCP examples.
Agent and workflow	Implemented composition surface over the uniform Session $\rightarrow$ Session contract, including a scripted multi-stage workflow.
Provider layer	Live-inference prerequisites in place; model quality and provider runs are out of scope for this report.
Memory plane	Intended runtime plane, not yet substantiated by a local module with examples and tests (see below).
Examples	A worked example set spanning lineage, backends, tools, streaming, usage, and multi-agent workflows.

The substantiated claims are deterministic and local. The session core—its ordered chunks, branching operations, usage accounting, and JSONL lineage export—is exercised by focused tests, as are local sandbox placement and the tool-dispatch path; tool and workflow behavior is further demonstrated by custom-tool, MCP, and scripted-workflow examples. The optional OpenSandbox backend and the LLM-provider layer are present but environment- or provider-dependent, and live model quality lies outside the scope of this report.

What the milestone demonstrates is not any individual surface but that they share a single object model. **Session** is the value that flows; backends determine where code and tools execute; tool calls enter the session as structured events rather than disappearing into an executor log; agents and workflows transform sessions without maintaining private transcript formats; and persistence with lineage export renders the resulting state inspectable outside the running process. This is the minimum structure an agent runtime requires to support branching, audit, and replay without rebuilding a separate observability system for each application.

8 Release Evidence and Evaluation Protocol

The release evaluation is audit-first. It scopes evidence to runtime claims rather than leaderboard claims: whether OpenRath’s runtime claims are backed by rebuildable evidence packets, whether each packet states its own boundary, and whether every visible claim is mapped into a claim ledger.

The claim ledger is the reviewer-facing contract. It currently classifies ten claims: five supported by operational packets, one partially supported, one supported only for prerequisites, one bibliography-backed positioning claim, one layout smoke claim, and one evidence-gated claim. This keeps the

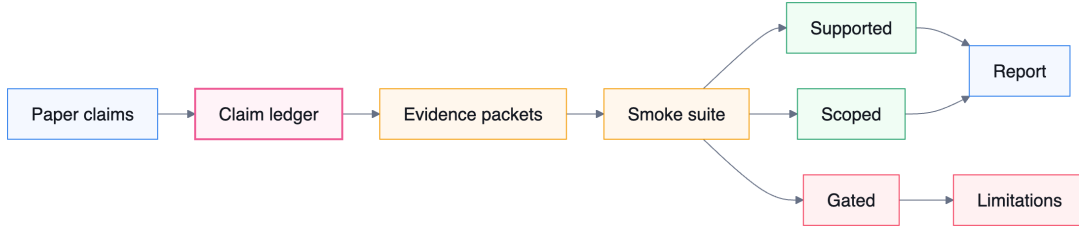


Figure 7: Claim-to-evidence protocol: report claims pass through a ledger, evidence packets, and a smoke suite before becoming supported text, scoped text, or explicit limitations.

report honest without making the paper read like an internal backlog: supported claims stay in the main thesis; evidence-gated claims stay visible and bounded, and framing claims are separated from empirical evidence.

An evidence packet is deliberately small. It contains the command that produced the run, a manifest, source and environment metadata, any session JSONL or tool logs, the generated output artifact, and a short summary of what the packet does and does not prove. This shape is easier to review than an informal run transcript because it gives a reader a direct path from paper claim to reproducible artifact. It also gives maintainers a practical release gate: a claim can move from evidence-gated to supported only when the corresponding packet runs under the documented environment.

Table 8: Current release evidence protocol.

Runtime claim	Current packet	Scope boundary
Session lineage is inspectable.	<code>lineage_export</code> : pass, deterministic.	Proves exported branch metadata, not branching quality.
Tool placement is auditable.	<code>local_sandbox</code> : pass; <code>opensandbox_optional</code> : skip.	Proves local placement evidence, not OpenSandbox parity.
Workflows compose session state.	<code>workflow_transcript</code> : pass, deterministic.	Proves composition shape, not live agent quality.
Implementation contracts hold for the focused subset.	<code>pytest_report</code> : pass.	Does not cover every live integration.
Provider prerequisites can be disclosed safely.	<code>live_provider_manifest</code> : pass, redacted.	Does not execute live inference.
Memory is scoped as a session-visible plane.	<code>memory_local</code> : skip.	Evidence-gated until source anchors exist.
Claim scope is tracked explicitly.	<code>claim_ledger</code> : pass, ten claims.	One evidence-gated claim remains: <code>memory_runtime_plane</code> .
Report layout is reviewable.	<code>visual_qa</code> and <code>layout_audit</code> : pass.	Visual smoke, not final human design approval.

This packet-first evaluation style is especially appropriate before broad benchmarks. Benchmarks such as curated coding suites [35] and broad general-assistant evaluations [47] are valuable, but they combine runtime semantics with model choice, prompt design, environment setup, reviewer scoring, and task distribution. For a runtime report, the first question is narrower: can the system preserve and expose the state needed to make those later evaluations meaningful? OpenRath’s current evidence protocol answers that narrower question before attempting comparative leaderboard claims.

The baseline and metric design follows the same principle. Follow-on comparisons should be organized by runtime shape rather than brand name: single-agent loop, multi-agent shared transcript, workflow/DAG runner, notebook or script baseline, sandboxed tool agent, and memory/RAG baseline. Metrics should track runtime correctness, provenance coverage, replayability, backend portability, efficiency, task quality, and control/safety events. Those metrics become results only when a runner emits comparable packets for OpenRath and each baseline.

9 Limitations

The limitations are stated as scope boundaries rather than apologies: they delimit what the report does and does not claim. The report substantiates a **Session**-centered runtime object with deterministic evidence for a narrow set of runtime claims. It does not claim broad benchmark superiority, a verified local-memory implementation, OpenSandbox availability, fully reproducible live-model outputs, or any safety property. Table 9 records each boundary, the current posture, and what a stronger claim would require.

Table 9: Scoped limitations for the current technical report.

Boundary	Current posture	Required before a stronger claim
Benchmarking	A deterministic smoke runner and evidence packets, not a broad baseline/metric benchmark.	Pinned workloads, baseline adapters, metrics, live-provider runs, and reviewer-scored artifacts.
Backend parity	Local-backend evidence passes; OpenSandbox is an explicit optional skip in this environment.	A backend capability matrix, a configured OpenSandbox packet, and documented mount, export, and failure behavior.
Memory	Memory remains an intended runtime plane; <code>memory_local</code> is evidence-gated because source anchors are absent.	Restored local-memory APIs, examples, and tests, then recall/commit quality evaluated separately from implementation existence.
Multi-agent control	Sessions expose branch, merge, tool, and lineage evidence, but do not constitute a policy layer.	Role permissions, tool authority, memory-commit gates, merge policy, and human-review requirements.
Safety	No safety property is claimed; tool use and interactive environments enlarge the attack surface, including the data/instruction confusion exploited by indirect prompt injection [49].	Evaluation against agent-, web-, and embodied-safety benchmarks [50–52], plus tool-authority limits and human-review gates.
Reproducibility	Packets support inspection and no-key replay for deterministic claims; live outputs remain provider- and environment-dependent.	Pinned source snapshots, provider manifests, sandbox images, cached external payloads, and disclosure of missing artifacts.

These boundaries are part of the release argument: they separate implemented runtime semantics from optional integrations, follow-on evaluation, and risks the report does not address. An item should leave this table only when a supporting evidence packet exists and the claim ledger maps the corresponding text to that artifact.

10 Conclusion

OpenRath’s contribution is deliberately narrow: it makes the state that agents operate on explicit. A multi-agent system is not only a prompt graph, a tool registry, a trace stream, or a benchmark

harness. It is a runtime in which conversation chunks, branch lineage, sandbox placement, tool effects, memory interactions, usage, artifacts, and replay evidence must remain connected.

Session is OpenRath’s proposed boundary for that runtime state. The programming model is compact because every major component either transforms a **Session**, annotates it, dispatches work through its placement, or emits evidence that can be inspected after the run. Because that evidence lives in the value the program already passes around rather than in a side channel reconstructed afterward, it stays available exactly when a reviewer needs it. This makes OpenRath complementary to graph runtimes, tracing SDKs, tool protocols, sandbox providers, and real-environment benchmarks rather than a replacement for them.

The current technical report therefore makes a scoped claim: deterministic runtime behavior can be reviewed through release packets today, while broader quality comparisons, memory-quality evaluation, and live-provider results belong in follow-on benchmark artifacts. The durable thesis is that reliable agent systems need a first-class runtime value, and OpenRath makes **Session** that value.

That thesis is also the practical standard for the next iteration of the project. New capabilities should enter the report only when they preserve the same boundary: they should transform a **Session**, attach evidence to a **Session**, or expose a backend effect through a **Session**. This keeps the system from becoming a collection of hidden side channels. It also keeps the report honest: implementation milestones, case studies, and evaluation claims can expand without changing the core argument that agent systems become easier to compose, debug, review, and evaluate when their runtime state is explicit.

If the last decade of deep learning made the tensor the value a network is built around, the next generation of agent systems needs the same move: a single runtime value that everything reads, transforms, and explains. OpenRath proposes that value is the **Session**.

References

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [2] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library, 2019. URL <https://arxiv.org/abs/1912.01703>.
- [3] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation, 2023. URL <https://arxiv.org/abs/2308.08155>.
- [4] LangChain. LangGraph: Persistence, 2025. URL <https://docs.langchain.com/oss/python/langgraph/persistence>.
- [5] OpenAI. OpenAI Agents SDK: Tracing, 2025. URL <https://openai.github.io/openai-agents-python/tracing/>.
- [6] Model Context Protocol. What is the Model Context Protocol (MCP)?, 2025. URL <https://modelcontextprotocol.io/docs/getting-started/intro>.
- [7] Anthropic. Introducing the Model Context Protocol, 2024. URL <https://www.anthropic.com/news/model-context-protocol>.
- [8] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, 2023. URL <https://arxiv.org/abs/2201.11903>.
- [9] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models, 2023. URL <https://arxiv.org/abs/2203.11171>.
- [10] Ehud Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham, Hofit Bata, Yoav Levine, Kevin Leyton-Brown, Dor Muhlgay, Noam Rozen, Erez Schwartz, Gal Shachaf, Shai Shalev-Shwartz, Amnon Shashua, and Moshe Tenenholtz. MRKL Systems: A Modular, Neuro-Symbolic Architecture that Combines Large Language Models, External Knowledge Sources and Discrete Reasoning, 2022. URL <https://arxiv.org/abs/2205.00445>.
- [11] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools, 2023. URL <https://arxiv.org/abs/2302.04761>.
- [12] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face, 2023. URL <https://arxiv.org/abs/2303.17580>.
- [13] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs, 2023. URL <https://arxiv.org/abs/2305.15334>.

- [14] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs, 2023. URL <https://arxiv.org/abs/2307.16789>.
- [15] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs, 2023. URL <https://arxiv.org/abs/2304.08244>.
- [16] Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. ToolAlpaca: Generalized Tool Learning for Language Models with 3000 Simulated Cases, 2023. URL <https://arxiv.org/abs/2306.05301>.
- [17] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models, 2023. URL <https://arxiv.org/abs/2305.10601>.
- [18] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society, 2023. URL <https://arxiv.org/abs/2303.17760>.
- [19] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework, 2024. URL <https://arxiv.org/abs/2308.00352>.
- [20] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative Agents for Software Development, 2024. URL <https://arxiv.org/abs/2307.07924>.
- [21] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors, 2023. URL <https://arxiv.org/abs/2308.10848>.
- [22] LangChain. LangGraph: Use Time Travel, 2025. URL <https://docs.langchain.com/oss/python/langgraph/use-time-travel>.
- [23] OpenAI. OpenAI Agents SDK: Agents, 2025. URL <https://openai.github.io/openai-agents-python/agents/>.
- [24] OpenTelemetry. Traces, 2025. URL <https://opentelemetry.io/docs/concepts/signals/traces/>.
- [25] OpenAPI Initiative. OpenAPI Specification Version 3.1.0, 2021. URL <https://swagger.io/specification/>.
- [26] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A System for Large-Scale Machine Learning, 2016. URL <https://arxiv.org/abs/1605.08695>.

- [27] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning, 2023. URL <https://arxiv.org/abs/2303.11366>.
- [28] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior, 2023. URL <https://arxiv.org/abs/2304.03442>.
- [29] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems, 2024. URL <https://arxiv.org/abs/2310.08560>.
- [30] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models, 2023. URL <https://arxiv.org/abs/2305.16291>.
- [31] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as Agents, 2025. URL <https://arxiv.org/abs/2308.03688>.
- [32] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains, 2024. URL <https://arxiv.org/abs/2406.12045>.
- [33] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- [34] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [35] OpenAI. SWE-bench Verified, 2024. URL <https://www.swebench.com/verified.html>.
- [36] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, et al. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- [37] Zhaoyang Chu, Jiarui Hu, Xingyu Jiang, Pengyu Zou, Han Li, Chao Peng, Peter O'Hearn, Earl T. Barr, Mark Harman, Federica Sarro, and He Ye. TerminalWorld: Benchmarking Agents on Real-World Terminal Tasks, 2026. URL <https://arxiv.org/abs/2605.22535>.
- [38] Sina Mavali, David Pape, Jonathan Evertz, Samira Abedini, Devansh Srivastav, Thorsten Eisenhofer, Sahar Abdelnabi, and Lea Schönherr. No More, No Less: Task Alignment in Terminal Agents, 2026. URL <https://arxiv.org/abs/2605.12233>.
- [39] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A Realistic Web Environment for Building Autonomous Agents, 2024. URL <https://arxiv.org/abs/2307.13854>.

- [40] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks, 2024. URL <https://arxiv.org/abs/2401.13649>.
- [41] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks?, 2024. URL <https://arxiv.org/abs/2403.07718>.
- [42] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments, 2024. URL <https://arxiv.org/abs/2404.07972>.
- [43] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents, 2023. URL <https://arxiv.org/abs/2207.01206>.
- [44] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a Generalist Agent for the Web, 2023. URL <https://arxiv.org/abs/2306.06070>.
- [45] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning, 2021. URL <https://arxiv.org/abs/2010.03768>.
- [46] Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. ScienceWorld: Is your Agent Smarter than a 5th Grader?, 2022. URL <https://arxiv.org/abs/2203.07540>.
- [47] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a Benchmark for General AI Assistants, 2023. URL <https://arxiv.org/abs/2311.12983>.
- [48] Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks, 2025. URL <https://arxiv.org/abs/2412.14161>.
- [49] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection, 2023. URL <https://arxiv.org/abs/2302.12173>.
- [50] Zhixin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. Agent-SafetyBench: Evaluating the Safety of LLM Agents, 2025. URL <https://arxiv.org/abs/2412.14470>.
- [51] Ada Defne Tur, Nicholas Meade, Xing Han Lù, Alejandra Zambrano, Arkil Patel, Esin Durmus, Spandana Gella, Karolina Stańczak, and Siva Reddy. SafeArena: Evaluating the Safety of Autonomous Web Agents, 2025. URL <https://arxiv.org/abs/2503.04957>.

- [52] Sheng Yin, Xianghe Pang, Yuanzhuo Ding, Menglan Chen, Yutong Bi, Yichen Xiong, Wenhao Huang, Zhen Xiang, Jing Shao, and Siheng Chen. SafeAgentBench: A Benchmark for Safe Task Planning of Embodied LLM Agents, 2025. URL <https://arxiv.org/abs/2412.13178>.

Appendix

A Case Studies

The case studies are used as scoped applicability arguments rather than benchmark results. Their purpose is to identify the kinds of workloads for which a **Session**-centered runtime model is intended, and to relate those workloads to the deterministic evidence already provided by the release packet suite. The current packets cover the runtime core: lineage export, local sandbox execution, scripted workflow composition, focused implementation tests, and explicit skips for evidence-gated memory and optional OpenSandbox support. The role of the case studies is therefore to connect these audited runtime claims to realistic workload patterns, without converting qualitative applicability into quantitative performance claims.

Table 10: Case-study coverage kept in the main report. Each row states what the workload demonstrates, not a measured leaderboard result.

Workload	Runtime point	Current evidence status
Repository editing	Multi-role planning, sandbox-bound tools, file artifacts, QA review, and lineage around a shared workspace.	Expressiveness coverage; promoted to a result only with a fixed repo seed and measured tests.
Research synthesis	Branch-specific context, compression, verifier roles, optional visual tooling, and final style head.	Expressiveness coverage; promoted to a result only with citation-faithfulness and compression-ablation packets.
Long-running coding	Persistence, resumability, budget control, context compression, sandbox continuity, and recovery after interruption.	Runtime coverage; promoted to a result only with seeded task packets and recovery measurements.
Memory-assisted workflow	Recall/commit should be visible as session operations rather than hidden prompt side effects.	Evidence-gated: audited source lacks current local-memory anchors.
Sandbox-isolated execution	Tool effects should carry backend placement, command/file/code logs, and cleanup status.	Local packet passes; optional OpenSandbox packet records a skip.

Repository editing and long-running coding provide the most direct motivation for durable and inspectable agent state. These settings include software-engineering benchmarks built from real GitHub issues [33], agent-computer interfaces for navigating, editing, and testing repositories [34], and company-style digital-worker tasks [48]. Research synthesis motivates branch-specific context, verifier roles, and controlled compression, since different branches may collect, filter, and restyle evidence before a final synthesis is produced. Memory-assisted workflows define the intended persistent-state plane, but remain scoped because the audited source tree does not yet provide current local-memory anchors. Sandbox-isolated execution is the strongest implementation-backed case: the local packet already records command execution, file effects, code execution, and cleanup status.